

Příprava na pohovor

AI architekt @ K&P; AI · 30 min call (15 prezentace + 15 Q&A)

Projekt: Job Fit & Salary Estimator · github.com/enrune/cv-salary-estimator

Tato příručka má 4 části: (0) co umět nazpaměť, (A) prezentační script slovo-od-slova, (B) technický cheat sheet, (C) mock otázky a odpovědi. Projdi celé aspoň 2× a demo si vyzkoušej naživo.

ČÁST 0 — Co se NAUČIT NAZPAMĚŤ

Tyhle čísla a fakta musíš odpovědět bez přemýšlení. Jsou základ celé obhajoby.

0.1 — Scoring vzorec (srdce projektu)

Seniority score (0-100) = vážený součet 4 faktorů:

$$\text{total} = 0.40 \times \text{experience} + 0.30 \times \text{skills} + 0.15 \times \text{education} + 0.15 \times \text{soft}$$

- Experience (40 %) — roky praxe, sigmoidní křivka (ne lineární): 2 roky→50, 5 let→74, 10 let→94
- Skills (30 %) — součet bodů za in-demand dovednosti (AWS=8, K8s=8, Python=6...) + breadth bonus
- Education (15 %) — SŠ=40, Bc=70, Mgr=85, PhD=100
- Soft (15 %) — leadership=25, mentoring=20, ownership=15... (z popisů rolí)

Proč sigmoid u praxe? Rozdíl mezi 1 a 5 lety je obrovský (junior→medior), mezi 10 a 15 lety marginální. Lineární by penalizoval seniory.

0.2 — Seniorita (z LET PRAXE, ne ze score!)

0-2 roky → junior | 2-6 let → medior | 6+ let → senior

Proč z praxe a ne ze score? Non-IT pozice (pokladník, lékař) mají strukturálně nižší skills score (méně IT dovedností v tabulce). Roky praxe jsou univerzální napříč obory. Score pak jen určuje POZICI v platovém rozmezí dané kategorie.

0.3 — Salary výpočet (5 kroků)

1. Načti tabulku (synthetic baseline + merge scraped data z jobs.cz)
2. Detekuj roli (LLM klasifikoval do whitelistu 52 kategorií)
3. Detekuj lokaci (Praha vs regiony — rozdíl ~20 %)
4. Lookup [min, max] range pro role × lokace × seniorita
5. Posun v range podle score (max ±12,5 %) + bonus za premium skills (max +20 %)

0.4 — Architektura: 6 kroků pipeline

ingest → parse → score → salary → explain → validate

- ingest (pdfplumber / python-docx) — PDF/DOCX → text
- parse (LLM) — text → Pydantic CV, self-healing retry při chybě
- score (heuristika, žádný LLM) — deterministické skóre
- salary (lookup + bonusy) — odhad mzdy
- explain (LLM) — české vysvětlení + 3+ doporučení
- validate — sanity checks (range, konzistence, počty)

Každý krok = čistá funkce. Pipeline jen orchestruje. Sdílená mezi UI / CLI / API.

0.5 — Tech stack (vyjmenovat plynule)

- Jádro: Python, Pydantic (structured outputs + validate)
- LLM: OpenRouter (1 klíč → Gemini/Claude/GPT), json_mode, retry
- Ingest: pdfplumber (PDF), python-docx (DOCX)
- Data: synthetic baseline + scrape jobs.cz (requests + BeautifulSoup4)
- UI/API: Streamlit (web), FastAPI (REST), CLI (argparse)

- Ostatní: loguru (logging), Docker

0.6 — Klíčová čísla

- 52 kategorií rolí (IT + non-IT)
- 306 platových rozmezí (52 rolí × 2 lokace × 3 seniority)
- 741 reálných inzerátů scrapnutých z jobs.cz
- 6 kroků pipeline, 12 modulů v src/
- 3 entry pointy (Streamlit / CLI / FastAPI)
- Typický run: ~5 000 tokenů, ~\$0.0008, ~10-14 sekund

ČÁST A — Prezentační script (15 minut)

Měj puštěný Streamlit + editor s kódem. Časy jsou orientační — drž se struktury.

► Minuty 0-2: Úvod + živé demo (HOOK)

Řekni:

„Dobrý den, děkuji za pozvání. Postavil jsem Job Fit & Salary Estimator — AI pipeline, která z životopisu v PDF nebo DOCX vrátí tři věci: seniority skóre 0 až 100, odhad tržní mzdy a české vysvětlení s konkrétními doporučeními, jak si polepšit o 30 procent. Pojďme rovnou na živé demo, ať vidíte, že to funguje.“

- Nahraj sample_cv_senior.docx → ukaž: Score 84, salary 142-202k, doporučení
- Pak nahraj jiné CV (pokladní / junior) → „a teď vidíte, že to funguje napříč obory, ne jen u IT“
- Zaškrtni Debug mode → „a tady je vidět, co se děje uvnitř — každý krok, prompty, tokeny, cena“

► Minuty 2-8: Architektura

Otevři src/pipeline.py a řekni:

„Pipeline má 6 kroků a každý je samostatný modul jako čistá funkce. Záměrně — abych je mohl testovat jednotlivě a sdílet stejnou logiku mezi webovým UI, příkazovou řádkou i REST API. Žádná duplicita.“

Projdi 6 kroků (ukazuj v kódu):

- „Ingest — z PDF nebo DOCX vytáhnu text. Použil jsem pdfplumber, protože je layout-aware.“
- „Parse — tady LLM převede volný text na strukturovaná data. Validuju to Pydanticem, a pokud model vrátí špatný JSON, mám self-healing retry — pošlu chybu zpátky do promptu a model to opraví.“
- „Score — čistá heuristika, žádný LLM. Schválně, protože je deterministická a vysvětlitelná.“
- „Salary — lookup v tabulce plus bonusy za žádané dovednosti.“
- „Explain — druhý LLM krok, generuje české vysvětlení a doporučení.“
- „Validate — sanity checks, aby se uživateli nezobrazil nesmysl.“

► Minuty 8-12: Klíčová rozhodnutí (NEJDŮLEŽITĚJŠÍ)

Tady ukážeš, že přemýšlíš jako architekt. Vysvětluj PROČ:

- „Proč heuristika a ne ML model? Nemám trénovací data — bez tisíců kalibrovaných CV by model overfittнул a nešlo by vysvětlit, proč dal score 84. Heuristika je transparentní a obhajitelná.“
- „Proč structured outputs s Pydanticem? LLM je nedeterministický. Pydantic mi garantuje, že výstup má správný tvar, jinak to padne hned a já to ošetřím retry.“
- „Jak jsem řešil data? Hybridně — ručně sestavená baseline tabulka pro 52 rolí, plus jsem scrapnul 741 reálných inzerátů z jobs.cz, plus heuristika. Funguje to i bez internetu.“
- „Seniorita z let praxe, ne ze score — aby to fungovalo i pro non-IT, kde je skills score přirozeně nižší. Lékař s 12 lety je senior, i když nemá AWS.“

► Minuty 12-15: Co bych zlepšil + závěr

„Kdybych měl víc času: přidal bych unit testy, víc datových zdrojů pro mzdy a embeddings pro lepší matching rolí. Vědomě jsem ale šel po funkčnosti — jak stálo v zadání, radši jednodušší a funkční než pokročilý a nedodělaný. Celé je to na GitHubu, spustitelné dvěma příkazy, a funguje i bez API klíče ve fallback módu. Děkuji, rád zodpovím dotazy.“

ČÁST B — Technický cheat sheet

Měj vytištěné vedle sebe. Rychlé odpovědi na technické otázky.

Q: Co když LLM vrátí nevalidní JSON?

A: Pydantic hodí ValidationError → zachytím → pošlu chybu zpět do promptu (retry s feedbackem) → pokud selže i podruhé, vyhodím ValueError s jasnou hláškou. Viz parse.py.

Q: Jak byste škáloval na 1000 CV?

A: Caching (mám @st.cache_data + @lru_cache na load tabulky), async LLM volání, batch processing, fronta úloh (Celery/RQ). CV se ořezává na 6000 znaků kvůli tokenům.

Q: Proč OpenRouter, ne přímo OpenAI?

A: Jeden klíč → 300+ modelů. Můžu A/B testovat Gemini vs Claude vs GPT bez změny kódu, jen env proměnnou. Plus fallback, když je model nedostupný.

Q: Jak měříte přesnost / kvalitu výstupu?

A: Sanity checks ve validate.py (8 pravidel: range, konzistence seniority/praxe, počet doporučení). Ručně jsem testoval 5 CV různých seniorit a oborů. Pro produkci by chtělo human-eval dataset.

Q: Nejslabší část řešení?

A: Salary data jsou odhad, ne autoritativní zdroj. Klasifikace role je keyword-based, ~30 % inzerátů spadne do "general". S víc daty bych použil embeddings pro fuzzy matching.

Q: Jak řešíte halucinace LLM?

A: Tři vrstvy: (1) nízká temperature 0.1 u extrakce, (2) Pydantic validace tvaru, (3) sanity checks na výsledná čísla. LLM nepočítá mzdu — tu počítá deterministická heuristika.

Q: Proč Streamlit a ne React frontend?

A: Rychlost vývoje — case study na 6-12 hodin. Streamlit dá funkční UI za hodinu. Pro produkci bych šel React + FastAPI backend (ten už mám).

Q: Jak funguje scraper?

A: requests + BeautifulSoup4 na jobs.cz, 35 query × 3 stránky. Regex na mzdu (toleranční na NBSP/ZWJ), keyword klasifikace role, agregace na percentily P25/P75. Slušný rate-limit 1.5s.

Q: Co je RAG a používáte ho tady?

A: RAG = Retrieval-Augmented Generation. Tady přímo ne, ale v projektu MindBase mám vlastní RAG (embedding pipeline, hybrid search cosine+BM25+RRF). Tady jde o structured extraction, ne retrieval.

Q: Jak by se to nasadilo do produkce?

A: FastAPI endpoint už mám → Docker kontejner → cloud (AWS/GCP). Přidal bych rate limiting, auth, monitoring (Prometheus), a DB pro ukládání výsledků. Logging už mám (loguru).

ČÁST C — Mock otázky & odpovědi

Projdi si je nahlas. Trénuj odpovědi, dokud nejdou plynule.

C.1 — Otázka, která MŮŽE PŘIJÍT: „Psal jste to sám?”

Buď upřímný, ale sebevědomý:

„Architekturu a všechna rozhodnutí jsem navrhoval já. Kód jsem psal s asistencí AI nástrojů — Claude, Cursor — což je dnes standardní vývojový workflow. Pro mě je důležité, že rozumím každému řádku a umím ho obhájit. Klidně mi otevřete libovolný soubor a projdu vám ho.”

Důležité: Tohle je legitimní. Moderní vývoj s AI je norma. Ale MUSÍŠ pak umět vysvětlit kód — proto je část 0 a B. Nikdy nelži, že jsi psal vše ručně.

C.2 — „Otevřete score.py a vysvětlete mi ho”

Otevři, ukaž experience score a řekni:

„Tady je vážený součet 4 složek. Váhy jsou nahoře jako konstanty, mám i assert že součet je 1.0. Funkce _experience_score mapuje roky na 0–100 sigmoidní křivkou — vidíte ty zlomy na 2, 5 a 10 letech. _skills_score sečte body za in-demand dovednosti z tohoto slovníku a přidá bonus za šířku. Všechno deterministické, žádný LLM, takže stejné CV dá vždy stejné skóre.”

C.3 — „Co je na vašem řešení nejlepší?”

„Tři věci: za prvé modularita — 6 nezávislých kroků sdílených mezi UI, CLI a API. Za druhé robustnost — structured outputs s Pydanticem a self-healing retry, takže to nespadne na špatném JSONu. A za třetí transparentnost — debug mód ukáže každý krok včetně promptů a ceny, a celé to funguje i bez LLM ve fallback módu.”

C.4 — „Jak byste to vylepšil, kdybyste měl měsíc?”

„Unit testy a CI. Více datových zdrojů pro mzdy — víc job boardů, ne jen jobs.cz. Embeddings pro sémantické matching rolí místo keyword matchingu. Human-eval dataset pro měření přesnosti. A produkční nasazení — Docker, auth, rate limiting, monitoring.”

C.5 — „Proč chcete dělat AI architekta zrovna u nás?”

„AI systémy stavím už teď ve volném čase — RAG, agenti, pipelines. Chci to dělat naplno a u startupu mě láká, že můžu stavět od základu a mít velkou zodpovědnost, což jste zmiňovali v zadání. Baví mě spojovat nástroje do funkčního celku, přesně to dělám v těchto projektech.”

C.6 — Když NĚCO NEVÍŠ

„To je dobrá otázka. Tuhle konkrétní část jsem řešil takhle [popiš], ale uznávám, že [alternativa] by mohla být lepší — rád si to nastuduji.”

Nikdy nefabuluj. Přiznat „tohle bych musel dohledat” je 10× lepší než vymyslet blbost, kterou recruiter prokoukne.

C.7 — OTÁZKY PRO NĚ (připrav si 3-4)

- „Jaký je váš současný AI stack a jaké modely používáte?”
- „Stavíte vlastní pipelines, nebo integrujete hotové nástroje?”
- „Jak velký je tým a jaká je zodpovědnost AI architekta?”
- „Co je teď vaše největší technická výzva?”
- „Jaké jsou další kroky výběrka?”

FINÁLNÍ CHECKLIST (den předem)

Technika

- streamlit run app.py funguje (API klíč nebo fallback připravený)
- 2-3 CV na ploše: sample_cv_senior, sample_cv_junior, sample_cv_pokladni
- VS Code otevřený se záložkami: pipeline.py, score.py, salary.py, parse.py
- GitHub repo otevřené v prohlížeči
- Kamera + mikrofon + sdílení obrazovky vyzkoušené
- Stabilní internet (kabel > wifi), tichá místnost
- Zavřené notifikace (Slack, Discord, mail) během sdílení

Hlava

- Část 0 (nazpaměť) projetá 2×
- Demo vyzkoušené 3× — jde ti hladce
- Prezentační script přečtený nahlas
- Jsi mezi 17 ze 40 — mají o tebe zájem, jdi s klidem

Zlaté pravidlo

Demo > teorie. Ukázat funkční věc je 10× silnější než o ní mluvit. Mluv pomalu a strukturovaně, radši méně ale jasně. Buď nadšený, ne arogantní. A pamatuj — už jsi prošel filtrem, tohle je jen potvrzení toho, co umíš. Hodně štěstí!